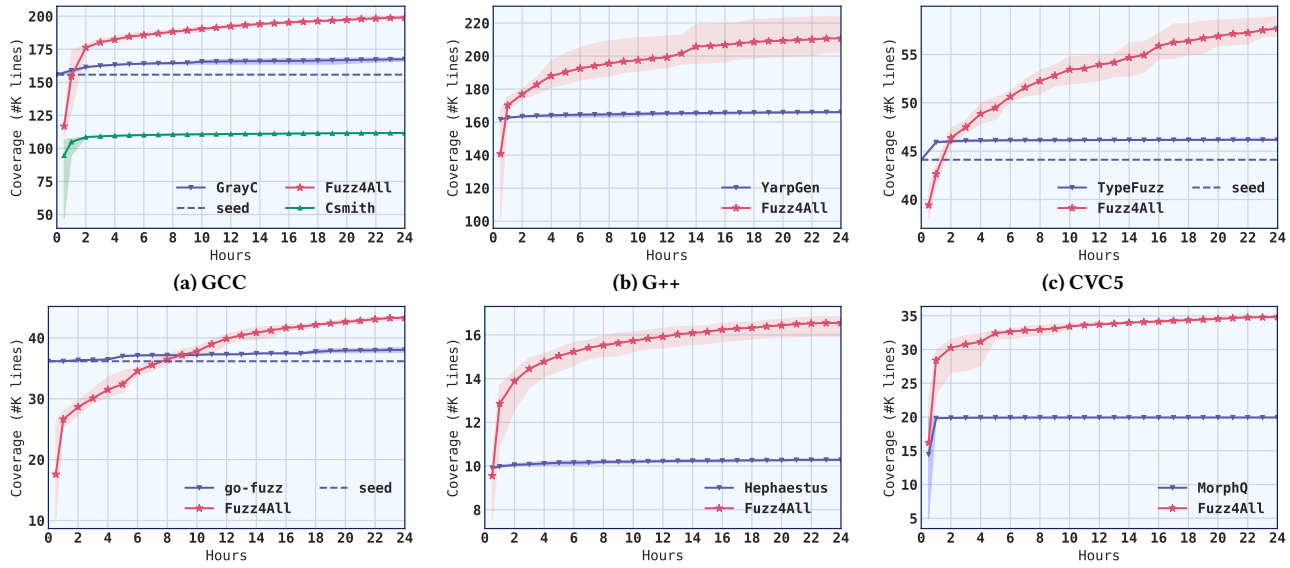

(a) GCC (b) G++ (c) CVC5

(d) Go (e) javac (f) Qiskit

Figure 4: Coverage trend of Fuzz4All against state-of-the-art fuzzers in a 24-hour fuzzing campaign.

that during each iteration of Fuzz4All's fuzzing loop, the original input prompt is updated with both a new example and a generation strategy (Section 3.2), nudging the LLM to generate new fuzzing inputs. We hypothesize that this allows Fuzz4All to effectively generate new and diverse fuzzing inputs even after a long period of fuzzing, leading to sustained coverage increase.

5.1.2 Generation Validity, Number, and Coverage. We examine the number of fuzzing inputs generated and their validity rate across our studied SUTs. In Table 2, Column "# programs" represents the number of unique inputs generated, "% valid" is the percentage of fuzzing inputs that are valid, and "Coverage" shows the final coverage obtained by each fuzzer along with the relative improvement over the best baseline. We first observe that almost all traditional fuzzing tools can achieve a very high validity rate apart from Hephaestus, which purposefully generates invalid code (focused on incorrect types) to check for miscompilation bugs. In contrast, Fuzz4All has a lower percentage of valid fuzzing inputs generated (56.0% average reduction compared to baseline tools). Furthermore, the raw number of fuzzing inputs generated by baseline tools are also much higher. By using an LLM as the generation engine, Fuzz4All is bottlenecked by GPU inference, leading to 43.0% fewer fuzzing inputs compared to traditional fuzzers.

In spite of the lower validity rate and number of fuzzing inputs, Fuzz4All generates much more diverse programs compared to traditional fuzzing tools, as evidenced by the high coverage obtained (+36.8% average increase). Additionally, even invalid code snippets that are close to valid can be useful for fuzzing, as they allow for finding bugs in the validation logic of the SUT. In Section 5.4, we further describe the various types of bugs detected by Fuzz4All, with both valid and invalid code snippets, to additionally showcase the benefit of generating diverse fuzzing inputs.

We note that Fuzz4All achieves a wide range of validity rates and numbers of fuzzing inputs across different SUTs. The number of fuzzing inputs varies across targets due to the varying cost to invoke the SUT after each fuzzing iteration for bug detection. Regarding validity rate, a general-purpose programming language, such as C, has a relatively lower validity rate compared to domain-specific languages, such as the SMT2 language used for SMT solvers. A more rigorous language, e.g., Go, which does not allow any declared but unused variables, has an even lower validity rate. We also observe a low validity rate for fuzzing quantum computing platforms. As quantum computing is an emerging area with its own set of library APIs, the generation LLM may not have seen as many examples of quantum programs during its training as for more established languages. Nevertheless, Fuzz4All is still able to leverage user-provided documentation to generate interesting fuzzing inputs, which leverage quantum library APIs and achieve an impressive coverage improvement (+75.6%) compared to the state-of-the-art fuzzer.

5.2 RQ2: Effectiveness of Targeted Fuzzing

We now evaluate the ability of Fuzz4All to perform targeted fuzzing, i.e., to generate fuzzing inputs that focus on a particular feature. For each target SUT and language, we test by targeting three different example features and compare them to the setup with general user input, as used for RQ1 (described in Section 4.3). These features are built-in libraries or functions/APIs (Go, C++ and Qiskit), language keywords (C and Java), and theories (SMT). The user input for the targeted fuzzing runs is documentation of the particular feature we are focusing on. Table 3 shows the results of targeted fuzzing as well as the default general fuzzing used in RQ1. Each column represents a targeted fuzzing run where we focus on one feature. The value in each cell shows the hit rate of the feature (Section 4.3) for a particular fuzzing run. We also include the coverage results obtained.

We observe that targeting a specific feature yields a high amount of fuzzing inputs that directly use the feature, with an average hit rate of 83.0%. This result demonstrates that Fuzz4All indeed performs targeted fuzzing by prompting the generation LLM with